EncryptionGuard v5 (Cloud-Light): Implementation Roadmap

Step-by-Step Guide for Cloud-First Development & AI Integration

Project: EncryptionGuard

Track: Razorpay AI Risk Manager (Coordinated Refund Abuse)

AI Provider: Xiaomi MiMo API

Hosting Strategy: Cloud-Light (No Local Docker Required)

This document provides a phased roadmap for building EncryptionGuard v5 using managed cloud services. This approach keeps your laptop fast and ensures the project is shareable via a public URL.

Phase 1: Managed Cloud Infrastructure

Goal: Set up the data layer without local resource load.

1.1 Managed Database Setup

Create accounts for the following free-tier services:

- **Supabase (PostgreSQL)**: Create a project and save the connection string for the system of record.
- **Neo4j Aura (Neo4j)**: Create a free instance for the relationship graph.
- **Upstash (Redis)**: Create a serverless Redis instance for velocity features.

1.2 Environment Configuration

Create a `.env` file (stored only in the backend and sandbox) containing:

- `SUPABASE_URL`, `SUPABASE_KEY`
- `NEO4J_URI`, `NEO4J_USER`, `NEO4J_PASSWORD`
- `UPSTASH_REDIS_URL`, `UPSTASH_REDIS_TOKEN`
- `MIMO_API_KEY`
- `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`

Phase 2: Data Engine & Evaluation Foundation

Goal: Generate synthetic data in the Manus Persistent Sandbox.

2.1 Persistent Sandbox Setup

Use a **Manus Persistent Sandbox** ($10/mo) as your heavy-duty Linux environment. This avoids laptop strain and keeps your work persistent across sessions.

2.2 Scenario Generator & Split Manifest

In the sandbox:

- Run the **Scenario Generator** to produce the labeled population (Normal, Single Abuse, Coordinated Rings).
- Create the **Split Manifest** (Train/Validation/Test) using time-aware and group-aware logic.
- Verify the **Stable Payment Token Simulation** logic.

Phase 3: ML Pipeline & Feature Library

Goal: Train the XGBoost model in the Persistent Sandbox.

3.1 Feature Engineering

Develop the shared Python `feature_library` to compute:

- Redis-based velocity features.
- Neo4j-based graph features (using the `reference_timestamp` to prevent leakage).

3.2 Model Training

- Train the **XGBoost** model using Bayesian optimization (Optuna).
- Save the trained model artifact, calibration object, and SHAP explainer configuration.
- Generate the **Model Card** for the v5 primary model.

Phase 4: Backend & API Integration (Manus WebDev)

Goal: Deploy the API and webhook receiver to Manus WebDev.

4.1 Initialize WebDev Project

Use `webdev_init_project` to create the application scaffold (`web-db-user` scaffold).

4.2 Webhook & API Implementation

- **Webhook Receiver**: Implement the FastAPI/Express endpoint for Razorpay webhooks.
- **Signature Verification**: Use the HMAC-SHA256 validation logic.
- **Xiaomi MiMo API Integration**: Connect the analyst assistant to the MiMo API (`gpt-5.4-mini`).
- **Policy Checker**: Implement the deterministic validator for AI outputs.

4.3 Asynchronous Processing

Configure background workers (using WebDev cron or the Persistent Sandbox) to handle Neo4j upserts and API enrichment.

Phase 5: Frontend Dashboard (Manus WebDev)

Goal: Build the investigator console.

5.1 Investigator Console

- **Alert Queue**: Ranked alerts with financial exposure estimates.
- **Case Detail**: Evidence timeline, SHAP explanations, and AI summaries.
- **Graph Visualization**: Interactive **Cytoscape.js** view connecting to Neo4j Aura.

5.2 Analyst Feedback Loop

Implement the buttons for confirmed abuse vs. legitimate cases to feed back into the retraining pipeline.

Phase 6: Evaluation, Demo & Handover

Goal: Finalize the winning submission.

6.1 Final Evaluation

Run the frozen held-out test in the Persistent Sandbox and export the metrics (PR-AUC, Ring Precision/Recall).

6.2 Demo Preparation

Host the project at `https://encryptionguard.manus.space`. Prepare the 5-minute recording showing the live webhook flow and AI-powered investigation.

6.3 Handover Checklist

- [ ] **Share Cloud Credentials**: Securely share the Supabase/Neo4j/Upstash access.
- [ ] **Export Sandbox State**: Ensure all scripts and model artifacts are in the repository.
- [ ] **Publish WebDev Project**: Ensure the latest dashboard is live.

References

[1]: Xiaomi MiMo API Open Platform

[2]: Supabase - Managed PostgreSQL

[3]: Neo4j Aura - Managed Neo4j

[4]: Upstash - Serverless Redis

[5]: Manus WebDev Documentation